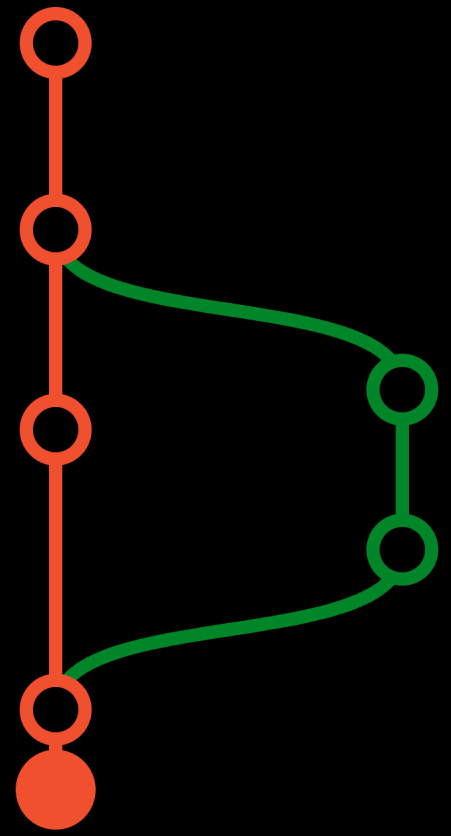
$ git init oficina

Oficina de Git & GitHub



Versionamento, colaboração e boas práticas para projetos de código

Maria Carolina Cabral de Almeida & Murilo de Andrade Souza · Engenharia de Software UPE · 22/10/2026

Roteiro da oficina

01	**Fundamentos**	Por que versionar código e a diferença entre Git e GitHub
02	**Git em 5 blocos**	Estados, fluxo e recuperação, boas práticas, branches e sincronização
03	**GitHub em 4 atos**	Pilares, triagem, IA e ChatOps, segurança
04	**Mão na massa**	Exercício colaborativo com toda a turma

Parte 01 — Fundamentos

Por que versionar código?

```
projeto_final_v2_AGORA_VAI_revisado.zip
```

Git não é GitHub

Git

Ferramenta de controle de versão que roda no seu computador. Guarda o histórico do projeto em commits.

`local · offline · linha de comando`

GitHub

Plataforma online que hospeda repositórios Git e adiciona colaboração: issues, pull requests e revisão.

`nuvem · equipe · interface web`

O Git funciona sem o GitHub; o GitHub não existe sem o Git.

Git

Do commit ao push

Cinco blocos: do que acontece dentro do Git até sincronizar com o mundo.

Bloco 1
Estados e áreas

Bloco 2
Fluxo e recuperação

Bloco 3
Boas práticas

Bloco 4
Branches e conflitos

Bloco 5
Sincronização

Bloco 1

Arquitetura, estados e áreas

Modified, Staged e Committed, e as três áreas locais

Três áreas, três estados



Arquivo novo começa como **untracked**; **git status** mostra o estado de cada um.

Bloco 2

Fluxo básico + recuperação

status, add e commit, cada um com seu desfazer: restore, reset e revert

Fluxo básico

```
$ git init                          # cria o repositório
$ git status                        # mostra o que mudou
$ git add index.html                # prepara o arquivo
$ git commit -m "primeiro commit"   # salva um snapshot
$ git log --oneline                 # mostra o histórico
a1b2c3d primeiro commit
```

Cada passo tem um desfazer

Situação	Avançar	Desfazer
Editou um arquivo	(só editar)	**`git restore app.js`** Descarta a mudança local
Preparou o arquivo	**`git add app.js`**	**`git restore --staged app.js`** Tira da staging, mantém a edição
Fez um commit local	**`git commit -m "..."`**	**`git reset --soft HEAD~1`** Desfaz o commit, mantém as mudanças
Commit já publicado	**`git push`**	**`git revert a1b2c3d`** Cria um novo commit que desfaz o antigo

reset ou revert?

git reset



A — B · C descartado (HEAD em B)

Volta o ponteiro e **reescreve o histórico**. Use só no que ainda é local.

git revert



A — B — C — C’ (C’ desfaz C)

Cria um commit novo e **preserva o histórico**. Seguro em branch compartilhada.

Bloco 3

Boas práticas

O que fica de fora com .gitignore e mensagens padronizadas com Conventional Commits

.gitignore: o que não entra

.gitignore

```
# dependências
node_modules/
# segredos e configs locais
.env
# arquivos gerados
dist/
*.log
# lixo do sistema
.DS_Store
```

✓ **Crie antes do 1º commit**

Evita versionar lixo e segredos desde o início

⚠ **Já foi commitado?**

Ignorar não basta: rode `git rm --cached arquivo`

📖 **Modelos prontos**

github.com/github/gitignore, um por linguagem

Conventional Commits

```
tipo(escopo): descrição no imperativo
```

feat:
Nova funcionalidade

fix:
Correção de bug

docs:
Só documentação

refactor:
Muda o código, não o comportamento

test:
Adiciona ou ajusta testes

chore:
Manutenção, dependências, configs

Evite
`"ajustes" · "agora vai" · "final2"`

Prefira
`fix(login): corrige senha vazia`

Bloco 4

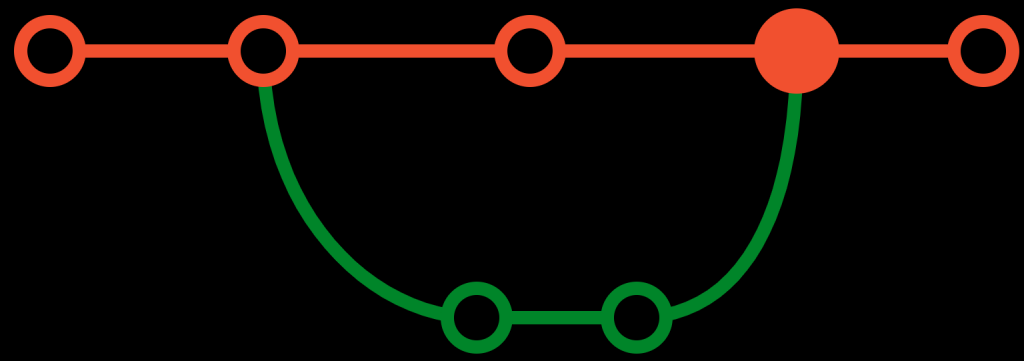
Branches, integração e conflitos

Criar branches, merge x rebase e como resolver um conflito

Branches: trabalho em paralelo

Um branch é uma linha do tempo separada. Você experimenta à vontade e só junta na main quando estiver pronto.

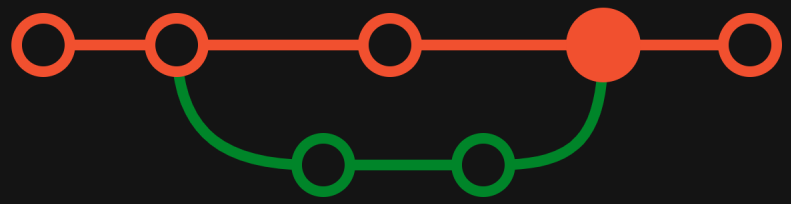
```
$ git switch -c nova-feature
$ git switch main
$ git merge nova-feature
```



• main • nova-feature ◉ merge

merge ou rebase?

git merge



```
$ git switch main && git merge feature
```

Preserva o histórico real e cria um **commit de merge**.

git rebase



```
$ git switch feature && git rebase main
```

Reaplica seus commits no topo da main: **histórico linear**.

Regra de ouro: **nunca faça rebase de um branch que outras pessoas já usam.**

Resolvendo um conflito

index.html

```
<<<<<<< HEAD
<h1>Bem-vindo à oficina</h1>
=======
<h1>Boas-vindas à oficina!</h1>
>>>>>>> nova-feature
```

em cima: sua versão · embaixo: a que chega

1. **git status** lista os arquivos em conflito
2. Escolha um lado ou combine os dois
3. Apague os marcadores <<< === >>>
4. **git add** + **git commit** para concluir

Bloco 5

Sincronização de repositórios

fetch para espiar, pull para baixar e unir, push para enviar

fetch, pull e push

remoto

origin

O repositório no GitHub

fetch



cópia local

origin/main

Retrato do remoto no último fetch

merge



seu branch

main

Onde você trabalha

git fetch

Espiar: baixa as novidades sem tocar no seu trabalho

git pull

Baixar e unir: fetch + merge, de uma vez

git push

Enviar: o caminho de volta, da main para o origin

Rotina de sincronização

```
$ git fetch origin                          # espiar
$ git log --oneline main..origin/main       # o que chegou?
$ git pull                                  # baixar e unir
$ git commit -am "feat: adiciona rodapé"    # seu trabalho
$ git push origin main                      # enviar
   a1b2c3d..9f8e7d6  main → main
```

GitHub

Do repositório à automação

Quatro atos: do básico da colaboração até IA, automação e segurança.

Ato 0

Os quatro pilares

Ato 1

Entrada e triagem

Ato 2

IA agêntica e ChatOps

Ato 3

Segurança e governança

Ato 0

Os quatro pilares

Repositório e README.md, Issues, Fork e Pull Request

Os quatro pilares do GitHub

Repositório

`+ README.md`

O projeto e seu histórico. O README é a porta de entrada.

Issues

`#42`

Onde nascem as tarefas: bugs, ideias e discussões.

Fork

`voce/projeto`

Sua cópia de um repositório alheio, para propor mudanças.

Pull Request

`PR #7`

O pedido para juntar suas mudanças, com revisão.

`Issue → Fork → Branch → Pull Request → Merge`

Ato 0 · Pilares | **Ato 1 · Triagem** | Ato 2 · IA e ChatOps | Ato 3 · Segurança

Ato 1

Entrada estruturada e triagem inteligente

Issue Forms + automação de triagem com GitHub Actions

Issue Forms: entrada estruturada

.github/ISSUE_TEMPLATE/bug.yml

```yaml
name: Relatar bug
labels: ["bug", "triagem"]
body:
  - type: dropdown
    id: area
    attributes:
      label: Área afetada
      options: [Frontend, Backend, Docs]
    validations:
      required: true
  - type: textarea
    id: passos
    attributes: { label: Como reproduzir }
```

- ✓ **Campos obrigatórios**
  Chega de issue vazia ou só "não funciona"
- ✓ **Dropdowns e checkboxes**
  Respostas padronizadas, fáceis de filtrar e automatizar
- ✓ **Labels automáticas**
  A issue já nasce com a etiqueta certa

Triagem automática com Actions

1

Issue aberta

Pelo formulário, com campos validados



2

Workflow dispara

```
on: issues
types: [opened]
```



3

Lê os campos

Área e gravidade, direto do corpo da issue



4

Classifica

Label, responsável e comentário de boas-vindas

Resultado: ninguém precisa ler cada issue nova para saber **quem cuida dela**.

Ato 2

IA agêntica e ChatOps

Copilot coding agent, revisão com IA, Copilot Autofix e slash commands em PRs

Copilot coding agent: da issue ao PR

1

Você

Escreva a issue

Contexto claro e critérios de aceite

2

Você

Atribua ao Copilot

Como faria com um colega de equipe

3

Copilot

Agente trabalha

Cria branch, commita e testa no Actions

4

Copilot

Abre o PR

Rascunho com o resumo do que mudou

5

Você

Revise e itere

Comente no PR; o agente ajusta

O agente não faz merge sozinho: o PR passa pela **mesma revisão e regras** de qualquer contribuição.

Revisão com IA: code review e Autofix

Copilot code review

Adicione o Copilot como revisor do PR. Ele comenta o diff e sugere mudanças que você aplica com um clique.

`Reviewers → Copilot`

Copilot Autofix

Quando o code scanning (CodeQL) acha uma vulnerabilidade, ele propõe a correção no próprio PR, com explicação.

`alerta → correção sugerida`

A IA sugere; quem aprova continua sendo **uma pessoa**.

ChatOps: slash commands no PR

aluna comentou

/preview

github-actions [bot]

Preview publicado: [url do preview]

build ok em 1m12s

A conversa do PR vira o painel de controle.

.github/workflows/preview.yml

```yaml
on:
  issue_comment:
    types: [created]
jobs:
  preview:
    if: >-
      github.event.issue.pull_request &&
      startsWith(github.event.comment.body, '/preview')
    runs-on: ubuntu-latest
    steps:
      - run: ./scripts/deploy-preview.sh
```

Ato 3

Segurança invisível e governança moderna

Secret scanning + push protection e Rulesets com Evaluate

Push protection em ação

```
$ git push origin main
remote: error: GH013: Repository rule violations found
remote: — GITHUB PUSH PROTECTION —
remote:  Push cannot contain secrets
remote:  GitHub Personal Access Token · config/.env:3
 ! [remote rejected] main → main (push declined)
```

Secret scanning

Varre o repositório e alerta sobre segredos que já foram expostos

Push protection

Bloqueia o push antes de o segredo entrar no histórico

Rulesets: regras com período de teste

Regras para a main

- Exigir PR com 1 aprovação
- Exigir status checks passando
- Bloquear force push e exclusão
- Exigir commits assinados

Enforcement status

Disabled

A regra existe, mas não faz nada

Evaluate

Registra quem violaria a regra, sem bloquear

Active

Bloqueia de verdade

Mão na massa

Caça aos Bots

Repositório: github.com/muliroZ/oficina-git-github

01

Dê uma estrela (⭐) no repositório

02

Abra uma Issue com template e encontre o 1° bot (Triagem)

03

Entre no Pull Request e comente **/deploy-preview** para achar o 2° bot (ChatOps)

```
$ git commit -m "fim da oficina"
```

Dúvidas?

Obrigado por participar!

github.com/muliroZ
github.com/CarolinaaCabral

LADES

Material de Estudo

Guia Git e GitHub: Do Júnior ao Sênior

QR Code do material:

Git & GitHub

LADES